



CATCH THE DOT – JUEGO INTERACTIVO CON CÁMARA

Proyecto de Procesamiento Digital de Imágenes y Visión Artificial

Autores: **Daniel Rua**, Profesor: **David Fernández**

Universidad de Antioquia – Facultad de Ingeniería Fecha: Octubre 2025

OBJETIVO GENERAL

Desarrollar un juego interactivo que utilice la cámara del computador como principal medio de entrada, aplicando conceptos de procesamiento digital de imágenes para detectar un objeto en tiempo real y generar una interacción lúdica.



PROBLEMA HUMANO A RESOLVER

La interacción tradicional con computadores mediante teclado y ratón limita la naturalidad y accesibilidad del usuario. Se busca crear una interfaz basada en visión por computadora que permita controlar un juego solo con el movimiento físico, mejorando la experiencia y fomentando el aprendizaje lúdico.



Interacción Tradicional

Limitada naturalidad y accesibilidad.



Visión por Computadora

Control del juego con movimiento físico.



Beneficios

Mejora la experiencia y fomenta el aprendizaje lúdico.

PROBLEMA TÉCNICO A RESOLVER

Implementar un sistema capaz de:

- 1 Captura de video en tiempo real.
- 2 Detectar un objeto mediante segmentación por color.
- 3 Calcular su posición en el plano de la imagen.
- 4 Evaluar colisiones con un objetivo en movimiento para generar la dinámica del juego.

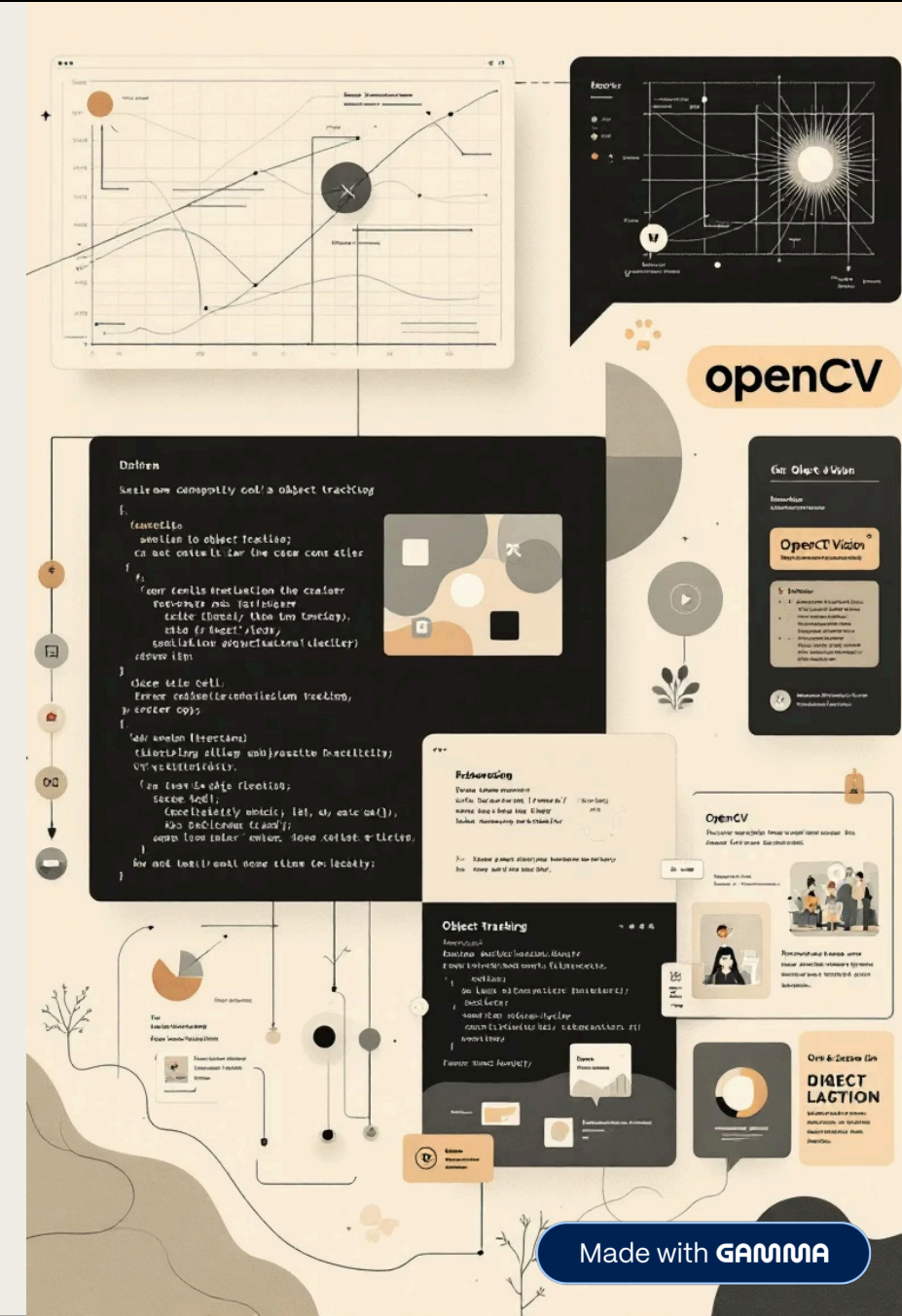
ANTECEDENTES Y REFERENCIAS

Antecedentes Clave

1. Juegos interactivos con reconocimiento visual usando OpenCV.
2. Interfaces basadas en segmentación por color y seguimiento de objetos.
3. Aplicaciones educativas con visión artificial.

Referencias

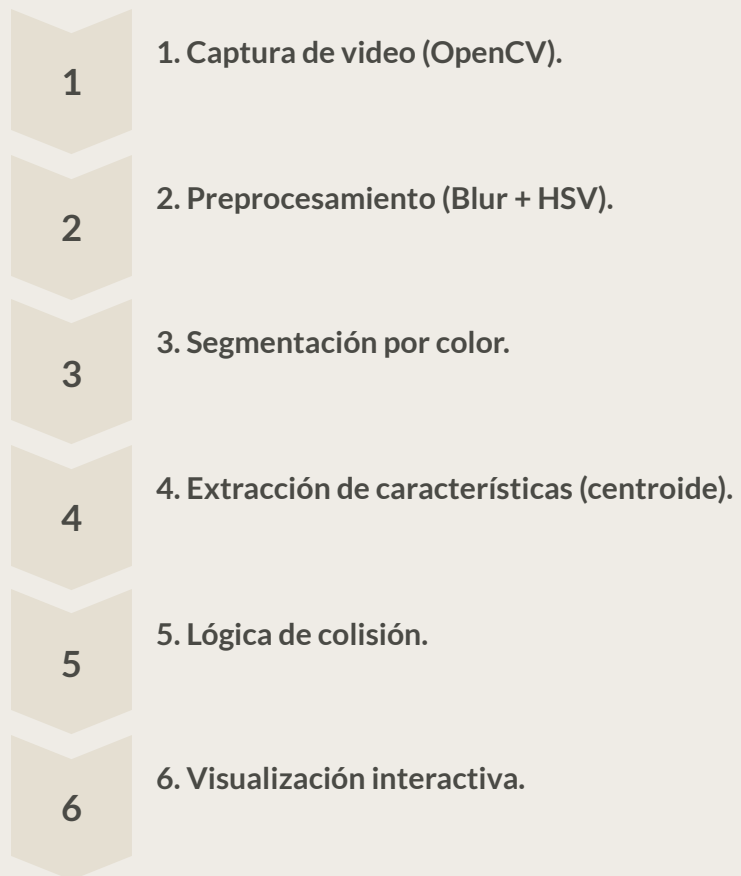
- OpenCV Documentation (opencv.org)
- LearnOpenCV: Color Detection and Object Tracking Tutorial
- Paper: "Vision-based Human-Computer Interaction: A Review" (IEEE, 2020)



ARQUITECTURA DEL SISTEMA

El sistema capta imágenes en tiempo real desde la cámara, las filtra para reducir ruido, convierte el espacio de color a HSV, segmenta el color elegido y calcula el centro del objeto detectado. Esa información se utiliza para determinar si el objeto coincide con la posición del objetivo del juego.

Etapas del flujo:



REPRESENTACIÓN MATEMÁTICA

Conversión de color:

$$HSV = f(BGR)$$

Máscara binaria:

$$M(x, y) = 1 \text{ si } H_{min} < H < H_{max}, S_{min} < S < S_{max}, V_{min} < V < V_{max}$$

$$M(x, y) = 0 \text{ en otro caso}$$

Centroide del objeto:

$$x_c = m_{10}/m_{00}$$

$$y_c = m_{01}/m_{00}$$

Colisión:

$$d = \sqrt{((x_c - x_t)^2 + (y_c - y_t)^2)}$$

Si $d < R \rightarrow$ impacto detectado

DIAGRAMA DE FLUJO



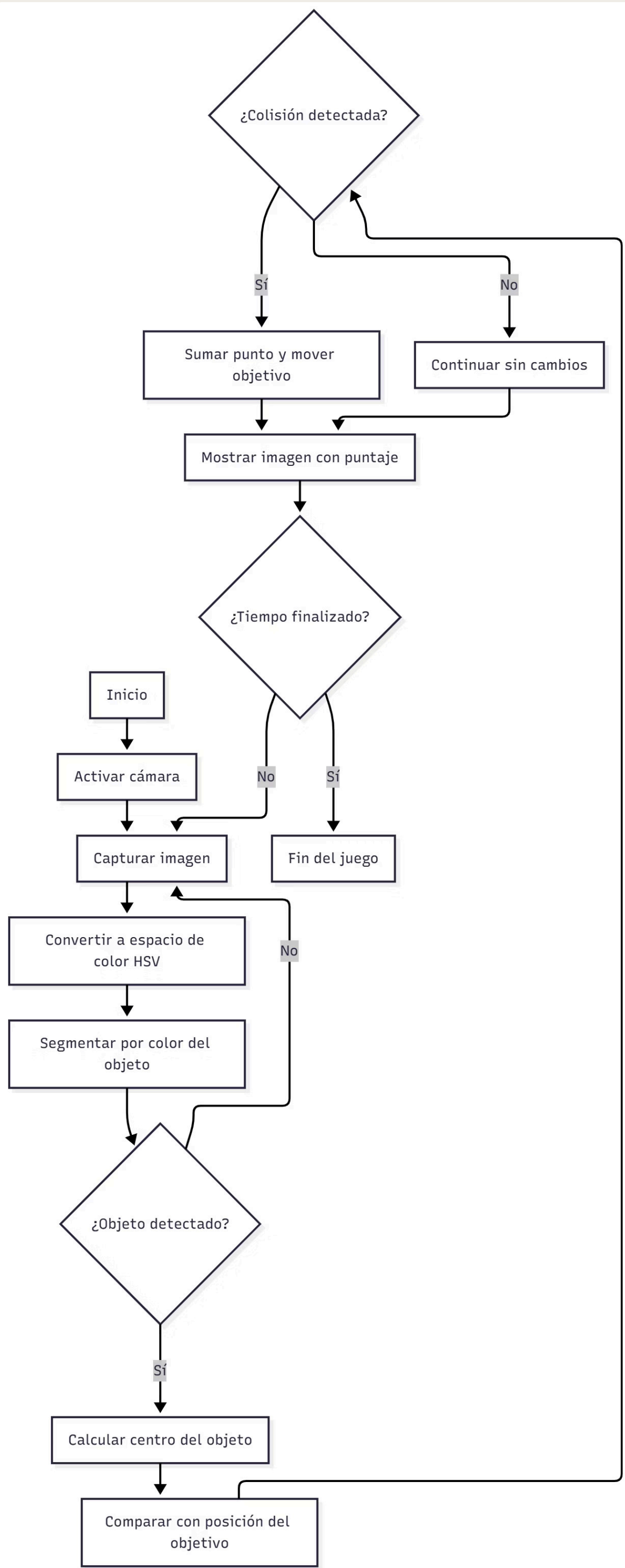
```
graph LR; A[Cámara] --> B[Preprocesado]; B --> C[HSV]; C --> D[Segmentación];
```

Cámara

Preprocesado

HSV

Segmentación



ALGORITMO PRINCIPAL (PSEUDOCÓDIGO)

Iniciar cámara

Mientras el juego esté activo:

- Leer frame

- Convertir a HSV

- Aplicar filtro por color

- Encontrar contornos

- Calcular centro del objeto

- Si colisión con objetivo:

 - Incrementar puntaje

 - Cambiar posición del objetivo

- Mostrar video con marcadores

Fin



ESTRUCTURA DEL CÓDIGO

1. Inicialización del sistema

Configuración de cámara y variables.

2. Funciones auxiliares

Creación de sliders HSV, cálculo de distancia y generación de objetivo.

3. Captura y preprocesamiento

Captura de imagen y aplicación de filtros.

4. Segmentación por color

Aplicación de máscara en espacio HSV.

5. Detección del objeto

Cálculo del centroide con momentos.

6. Lógica del juego

Detección de colisiones.

7. Visualización del resultado

Cámara, marcador, puntaje y tiempo.